

Stock Challenge

Full Project Documentation and README Reference

Backend: NestJS + PostgreSQL + Prisma + Redis + WebSockets

Frontend: React + Vite + TypeScript + Tailwind CSS

This document explains setup, architecture, roles, trading workflow, testing, and troubleshooting for the Stock Challenge project.

Important: This is a stock trading simulation. It is not connected to a licensed exchange or broker.

Stock Challenge

A full-stack real-time stock trading challenge application built with:

- Backend: NestJS, PostgreSQL, Prisma, Redis, WebSockets
- Frontend: React, Vite, TypeScript, Tailwind CSS
- Database ORM: Prisma 6.x
- Realtime: Socket.IO WebSockets
- Authentication: JWT access/refresh tokens with email verification

Important: This project is a stock-market simulation and order-matching system. It is not connected to a licensed exchange or broker. Do not use it for real-money public trading without proper security, compliance, KYC, audit, legal review, and financial licensing.

1. Main Features

Authentication

- Register account
- Real email verification flow
- Login with JWT access token and refresh token
- Protected API routes
- Active/deactivated user checks
- Admin/User roles

Admin Features

- Admin dashboard with system summary charts
- View all users
- Activate/deactivate users
- Create companies
- Approve/reject/suspend companies
- Create stocks for approved companies
- View all orders in the system

User Features

- Dashboard with live balance refresh
- View market/stocks
- View stocks listed for sale by other users
- Buy stocks from selling listings
- View portfolio holdings
- Sell owned stocks from portfolio using popup form
- View own orders and status
- Cancel open orders

- View completed trades

Trading Features

- User-to-user stock selling flow
- Limit buy orders
- Limit sell orders
- Order book by stock symbol
- Matching engine
- Buyer balance locking
- Seller stock locking
- Portfolio updates after completed trade
- Seller receives money after completed sale
- Sold-out stock disappears from seller portfolio
- WebSocket live updates for order book, trades, and stock price

2. Final Trading Flow

This is the current intended flow:

```
User A owns stock in Portfolio
User A clicks Sell
Popup asks sell quantity and price
Sell order is created immediately as OPEN
Sell order appears on Selling Stocks page
User B clicks Buy on that sell listing
Buy order is created immediately
Matching engine matches BUY and SELL
Trade is created
Seller balance increases
Buyer balance decreases
Buyer portfolio increases
Seller portfolio quantity decreases
If seller quantity becomes 0, stock disappears from seller portfolio
Both users see the trade in My Trades
```

There is no admin approval for buy/sell orders in the final flow. Admin can view orders, but users trade directly through the matching engine.

3. Project Folder Structure

```
stock-challenge/
  backend/
    prisma/
      schema.prisma
      migrations/
    src/
      auth/
      balances/
      companies/
      common/
      mail/
      market-data/
      matching-engine/
      orders/
      portfolios/
      prisma/
      stocks/
      trades/
      users/
    .env
```

```
package.json

frontend/
  src/
    components/
    lib/
    pages/
    types/
  .env
  package.json

docker-compose.yml
```

4. Prerequisites

Install these before running the project:

- Node.js 22.x
- npm
- Docker Desktop
- PostgreSQL container through Docker
- Redis container through Docker
- VS Code

Check versions:

```
node -v
npm -v
docker -v
```

5. Docker Setup

From project root:

```
cd C:\Users\DELL\stock-challenge
docker compose up -d
docker ps
```

Expected containers:

```
stock_postgres
stock_redis
```

If you need a clean database:

```
docker compose down -v
docker compose up -d
```

6. Backend Environment

Create this file:

```
backend/.env
```

Example:

```
DATABASE_URL="postgresql://stock_user:stock_password@localhost:5433/stock_challenge?schema=public"

JWT_ACCESS_SECRET="change_this_access_secret"
JWT_REFRESH_SECRET="change_this_refresh_secret"

ACCESS_TOKEN_EXPIRES_IN="15m"
REFRESH_TOKEN_EXPIRES_IN="7d"
APP_URL="http://localhost:5173"
```

```

BACKEND_URL="http://localhost:3000"

REDIS_HOST="localhost"
REDIS_PORT="6379"

SMTP_HOST="smtp.mailtrap.io"
SMTP_PORT="2525"
SMTP_USER="your_mailtrap_user"
SMTP_PASS="your_mailtrap_pass"
SMTP_FROM="Stock Challenge <noreply@stockchallenge.local>"

INITIAL_USER_BALANCE="1000000"

```

If your Docker PostgreSQL is mapped to port 5432 instead of 5433, change the `DATABASE\_URL` port to `5432`.

7. Backend Setup

```

cd C:\Users\DELL\stock-challenge\backend
npm install
npx prisma validate
npx prisma generate
npx prisma migrate dev
npm run start:dev

```

Backend runs on:

```
http://localhost:3000
```

Swagger API documentation:

```
http://localhost:3000/api/docs
```

Run backend checks:

```

npm run format
npm run lint
npm run start:dev

```

8. Prisma Notes

The project uses Prisma 6.x. In `schema.prisma`, this is correct:

```

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

```

If VS Code shows this warning:

```
The datasource property url is no longer supported in schema files
```

but terminal shows:

```

prisma      : 6.19.3
@prisma/client : 6.19.3

```

then it is a VS Code Prisma extension warning caused by Prisma 7 validation. Keep the `url` line for Prisma 6.

9. Frontend Environment

Create:

```
frontend/.env
```

Add:

```
VITE_API_URL=http://localhost:3000
```

10. Frontend Setup

```
cd C:\Users\DELL\stock-challenge\frontend
npm install
npm run lint
npm run build
npm run dev
```

Frontend runs on:

`http://localhost:5173`

11. Backend Modules and Endpoints

Auth Module

- `POST /auth/register`
- `POST /auth/login`
- `GET /auth/verify-email`
- `POST /auth/resend-verification`
- `GET /auth/me`

Users Module

- `GET /users`
- `GET /users/:id`
- `PATCH /users/:id/status`

Companies Module

- `POST /companies`
- `GET /companies`
- `GET /companies/:id`
- `GET /companies/mine`
- `PATCH /companies/:id/approve`
- `PATCH /companies/:id/reject`
- `PATCH /companies/:id/suspend`

Stocks Module

- `POST /stocks`
- `GET /stocks`
- `GET /stocks/active`
- `GET /stocks/symbol/:symbol`
- `PATCH /stocks/:id/status`

Balances Module

- `GET /balances/me`

- `POST /balances/deposit`

Portfolios Module

- `GET /portfolios/me`
- `GET /portfolios/summary`
- `GET /portfolios/:symbol`

Orders Module

- `POST /orders`
- `GET /orders/me`
- `GET /orders/:id`
- `DELETE /orders/:id/cancel`
- `GET /orders/orderbook/:symbol`
- `GET /orders/admin/all`

Trades Module

- `GET /trades/me`
- `GET /trades/recent/:symbol`

12. Frontend Pages

Public Pages

- `/login` - login page
- `/register` - account registration
- `/verify-email` - email verification result

User Pages

- `/dashboard` - balance and account summary
- `/market` - active listed stocks
- `/selling-stocks` - all open sell orders from users
- `/portfolio` - owned stocks and sell popup
- `/orders` - own order history/status
- `/trades` - completed trades
- `/stocks/:symbol` - stock detail/order book/trading screen

Admin Pages

- `/admin/dashboard` - system overview charts
- `/admin/users` - activate/deactivate users
- `/admin/companies` - approve/reject/suspend companies
- `/admin/stocks` - create/list stocks
- `/admin/orders` - view all orders

13. User Role Rules

Admin

Admin can:

- View system dashboard
- Manage users
- Approve/reject/suspend companies
- Create stocks
- View all orders

Admin should not use normal user tabs in the sidebar.

User

User can:

- View market
- Buy stocks from Selling Stocks page
- View portfolio
- Sell owned stocks from Portfolio
- Cancel own open orders
- View own trades

User should not see admin tabs.

14. Order and Trade Rules

Sell Order

When a user sells stock from Portfolio:

```
Portfolio quantity decreases
Portfolio lockedQuantity increases
Sell order status becomes OPEN
Sell listing appears on Selling Stocks page
```

No money is added yet. Money is added only when another user buys and a trade completes.

Buy Order

When a user buys from Selling Stocks:

```
Buyer availableLkr decreases
Buyer lockedLkr increases
Buy order status becomes OPEN
Matching engine tries to match the sell order
```

If matched:

```
Trade is created
Buyer portfolio increases
Buyer lockedLkr decreases
Seller lockedQuantity decreases
Seller availableLkr increases
Seller totalLkr increases
If seller has 0 quantity and 0 lockedQuantity, portfolio row is deleted
```

15. Matching Conditions

A trade happens when:

```
Same stock symbol
BUY price >= SELL price
BUY and SELL belong to different users
Both orders are OPEN or PARTIALLY_FILLED
Quantities overlap
```

Example:

```
SELL SPC at 100 quantity 5
BUY SPC at 100 quantity 2
Result: trade for quantity 2 at price 100
```

16. WebSocket Events

The backend sends live updates through Socket.IO:

```
joinStockRoom
leaveStockRoom
orderbook:update
trade:new
stock:price:update
```

Frontend uses these on stock detail pages to update order book and recent trades.

17. Full Test Flow

Step 1 - Start Docker

```
cd C:\Users\DELL\stock-challenge
docker compose up -d
```

Step 2 - Start Backend

```
cd C:\Users\DELL\stock-challenge\backend
npm run start:dev
```

Step 3 - Start Frontend

```
cd C:\Users\DELL\stock-challenge\frontend
npm run dev
```

Step 4 - Create Admin

1. Register `admin@test.com`. 2. Verify email from backend terminal link. 3. Open Prisma Studio:

```
npx prisma studio
```

4. Change admin user:

```
role = ADMIN
isEmailVerified = true
isActive = true
```

Step 5 - Create Company and Stock

1. Login as admin. 2. Go to `/admin/companies`. 3. Create company `SPC`. 4. Approve company. 5. Go to `/admin/stocks`. 6. Create stock `SPC` with initial shares and price.

Step 6 - Get First Seller Stock

The admin receives initial shares when the stock is created. For user-to-user selling, a normal user must first buy shares from an existing seller or you can use an inventory/admin account for testing.

Step 7 - Sell Owned Stock

1. Login as a user who owns stock. 2. Go to `/portfolio`. 3. Click Sell. 4. Enter quantity and price. 5. Confirm. 6. Sell order appears on `/selling-stocks`.

Step 8 - Buy Listed Stock

1. Login as another user. 2. Go to `/selling-stocks`. 3. Click Buy. 4. Confirm. 5. Matching engine executes the trade if prices match.

Step 9 - Verify Results

Seller:

```
Balance availableLkr increased
Stock quantity decreased
Sold-out stock removed from portfolio
Trade visible in My Trades
```

Buyer:

```
Balance decreased
Stock appears in portfolio
Trade visible in My Trades
```

18. Troubleshooting

P1000 Authentication Failed

Check `DATABASE\_URL`, Docker container password, and port.

```
docker ps
docker exec -it stock_postgres psql -U stock_user -d stock_challenge
```

Prisma Schema Red Warning in VS Code

If terminal says Prisma 6.19.3 and `npx prisma validate` passes, ignore or disable the VS Code Prisma extension for this workspace.

My Trades Empty

Trades only appear after a completed match. An OPEN order is not a trade.

Check:

```
Same symbol
BUY price >= SELL price
Opposite order exists
Different buyer and seller
```

Portfolio Not Updating

Portfolio updates only after a completed trade. Listing stock for sale only locks stock; it does not add money.

User Cannot Sell

User can sell only if `portfolio.quantity > 0`.

Seller Money Not Added

Money is added only after another user buys and the trade completes. Listing stock for sale does not add money.

Stock Still Visible After Sale

Refresh portfolio. If quantity and lockedQuantity are zero, backend deletes/hides the portfolio row.

CORS Error

Backend `main.ts` must include:

```
app.enableCors({
  origin: ['http://localhost:5173'],
  credentials: true,
});
```

Email Verification Not Sending

If Mailtrap is not configured, the backend logs the verification URL in the terminal.

19. Security Notes

Before real production use, add:

- Strong secrets in environment variables
- HTTPS only
- Password reset flow
- Refresh token rotation
- Rate limiting for login/register
- Request validation everywhere
- Audit logs for admin actions
- Input/output sanitization
- Proper permissions for every endpoint
- KYC/AML and legal compliance if real money is involved
- Real payment gateway integration
- Production monitoring and logs

20. Final Submission Checklist

```
Backend starts without errors
Frontend starts without errors
Docker PostgreSQL and Redis running
Swagger opens
Register works
Email verification works
Login works
Admin/user sidebar separation works
Admin can manage users
Admin can approve companies
Admin can create stocks
User can buy stock from Selling Stocks
User can sell owned stock from Portfolio
Seller money increases after sale
```

Buyer portfolio increases after purchase
Sold-out stock disappears from seller portfolio if quantity is zero
My Trades shows completed trades
Order book updates
WebSocket updates work
README and PDF documentation included

21. Useful Commands

```
# Project root
cd C:\Users\DELL\stock-challenge

docker compose up -d
docker compose down -v

# Backend
cd C:\Users\DELL\stock-challenge\backend
npx prisma validate
npx prisma generate
npx prisma migrate dev
npx prisma studio
npm run format
npm run lint
npm run start:dev

# Frontend
cd C:\Users\DELL\stock-challenge\frontend
npm run lint
npm run build
npm run dev
```